Campus Equipment Loan API

Stage 1 tutorial · CSC 640 · Minh Son Le

Borrowers browse equipment, check it out, and return it. Staff manage catalog items and can review and close loans. Requests pass through NGINX to PHP-FPM and use PDO to read and write MySQL.

Start and test

```
cd code/stage1
bash scripts/start.sh
```

Open `http://127.0.0.1:8640/test.html`. Select an endpoint, edit its JSON body, enter a record ID if needed, and click **Send request**.

For synthetic staff and borrower accounts, run the fixture reset in the README before testing. It clears and replaces records in the isolated course database.

Sign up and sign in

- **POST /auth/register** takes `name`, `email`, and `password`. Returns **201** with the new borrower's ID and profile. Invalid fields yield **400**, duplicate email **409**.
- **POST /auth/login** takes `email` and `password`. Returns **200** with `data.token` and `data.token_type`. Wrong credentials yield **401**.
- Both routes are public. Paths on these slides have the prefix `/api/v1`. A successful login token can be reused with the button on the test page.

Browse equipment

- **GET /equipment** returns **200** with a list in `data`. Optional query parameter `available=true` or `false` filters results. Other values yield **400**.
- **GET /equipment/{id}** returns **200** with one item in `data`, or **404** for an unknown ID.
- Both are public. An item contains its numeric ID, name, description, and Boolean availability. An active loan makes an item unavailable.

Edit the catalog

Send `Authorization: Bearer <staff token>` on each request.

- **POST /equipment** takes `name` and optional `description`. Returns **201** with the created item.
- **PUT /equipment/{id}** takes the same fields. Returns **200** with the updated item, or **404** if absent.
- **DELETE /equipment/{id}** needs only the ID. Returns **204** with an empty body, **404** if absent, or **409** if the item has loan history.

Invalid fields yield **400**. Missing or invalid tokens yield **401**. Borrower tokens yield **403**.

Check out and browse loans

- **POST /loans** takes a JSON integer `equipment_id` and a borrower token. Returns **201** with the loan and `returned_at: null`. Bad input yields **400**, unknown item **404**, and an existing active loan **409**.
- **GET /loans** needs a bearer token and no body. Returns **200** with the borrower's loans or all loans for staff.
- A loan includes `id`, `user_id`, `equipment_id`, `borrowed_at` and `returned_at`. Both routes return **401** for missing or invalid credentials.

Return a loan

PUT /loans/{id}/return takes the loan ID in the path and no body. The borrower who owns the loan or a staff member can close it.

200 returns the loan with its return timestamp. A different borrower gets **403**, an unknown loan gets **404**, and repeating a return gets **409**. Missing or invalid bearer tokens get **401**.

Observed success and failure

```
GET /api/v1/equipment/1001
HTTP 200
{"data":{"id":1001,"name":"Tripod",
 "description":"Available fixture","available":true}}

PUT /api/v1/loans/2001/return
Authorization: Bearer <borrower 102 fixture token>
HTTP 403
{"error":{"code":"forbidden","message":
 "Only the borrower or staff can return this loan."}}
```

The browser test reproduced both responses. The integrity check compared the loan list before and after the forbidden request.

Reproduce the results

From the repository root, after starting Stage 1 and resetting the fixtures:

```
python3 code/tests/acceptance.py \
  --base-url http://127.0.0.1:8640 --stage stage1 \
  --output docs/results/stage1.json
python3 code/tests/stage1-integrity.py
```

The recorded run passed **44/44** assertions. The integrity check sent two simultaneous checkout requests for one item and observed one **201**, one **409**, and one active loan.

References and assistance

- ASE 230 Module 1: *2. Setting up NGINX for PHP.pdf*, p. 1 onward, for NGINX/PHP-FPM setup.
- ASE 230 Module 1: *3. Connecting REST API Server to MySQL with PDO.pdf*, p. 1 onward, for PDO and database-backed endpoints.
- ASE 230 Module 1: *4. Testing REST API Server.pdf*, p. 1 onward, for API verification.
- OpenAI Codex helped implement the API and test page, run the checks, and draft this tutorial. Examples use the project's synthetic fixtures.